Lecture 2: Resource accounting

Building Large Foundation Models from Scratch · WS 2026/27

Asst. Prof. Dr. Martin Pawelczyk

Faculty of Computer Science, University of Vienna

Tue 6 Oct 2026

Where we are

Admin, 2 minutes

- Registered on Moodle? Hands up if you are still not on Moodle.
- Clone repo, `pytest` run. Did you start thinking about the tokenizer?
- Colab: does a GPU runtime start for you? Lab 2 builds on it.

Today (PC room: have PyTorch ready)

1. Tensors, bytes and precision
2. Counting FLOPs
3. Memory during training
4. Arithmetic intensity: why FLOPs are not time
5. What you need for Assignment 1
6. Wrap-up

Motivating Questions

How long does it take to train a 70B model on 15T tokens with 1024 H100s?

What is the largest model you can train on 8 H100s with AdamW?

Motivating Questions

How long does it take to train a 70B model on 15T tokens with 1024 H100s?

```
N, D = 70e9, 15e12
total_flops = 6 * N * D  # 6.3e24
```

What is the largest model you can train on 8 H100s with AdamW?

Motivating Questions

How long does it take to train a 70B model on 15T tokens with 1024 H100s?

```
N, D = 70e9, 15e12
total_flops = 6 * N * D  # 6.3e24
peak = 1979e12 / 2  # "sparse" / 2
mfu = 0.5
per_day = peak * mfu * 1024 * 86400
```

What is the largest model you can train on 8 H100s with AdamW?

Motivating Questions

How long does it take to train a 70B model on 15T tokens with 1024 H100s?

```
N, D = 70e9, 15e12
total_flops = 6 * N * D  # 6.3e24
peak = 1979e12 / 2  # "sparse" / 2
mfu = 0.5
per_day = peak * mfu * 1024 * 86400
days = total_flops / per_day  # 144
```

Five months on a thousand GPUs. Llama 3 70B took about that, at a lower MFU.

What is the largest model you can train on 8 H100s with AdamW?

Motivating Questions

How long does it take to train a 70B model on 15T tokens with 1024 H100s?

```
N, D = 70e9, 15e12
total_flops = 6 * N * D  # 6.3e24
peak = 1979e12 / 2  # "sparse" / 2
mfu = 0.5
per_day = peak * mfu * 1024 * 86400
days = total_flops / per_day  # 144
```

Five months on a thousand GPUs. Llama 3 70B took about that, at a lower MFU.

What is the largest model you can train on 8 H100s with AdamW?

```
gpu_bytes = 8 * 80e9
# params 2 + grads 2 + m 4 + v 4
bytes_per_param = 2 + 2 + 4 + 4
```

Motivating Questions

How long does it take to train a 70B model on 15T tokens with 1024 H100s?

```
N, D = 70e9, 15e12
total_flops = 6 * N * D  # 6.3e24
peak = 1979e12 / 2  # "sparse" / 2
mfu = 0.5
per_day = peak * mfu * 1024 * 86400
days = total_flops / per_day  # 144
```

Five months on a thousand GPUs. Llama 3 70B took about that, at a lower MFU.

What is the largest model you can train on 8 H100s with AdamW?

```
gpu_bytes = 8 * 80e9
# params 2 + grads 2 + m 4 + v 4
bytes_per_param = 2 + 2 + 4 + 4
n_params = gpu_bytes / bytes_per_param
# -> 4e10
```

40B parameters, an *upper bound*: activations are not counted, and they depend on batch size and sequence length.

Motivating Questions

How long does it take to train a 70B model on 15T tokens with 1024 H100s?

```
N, D = 70e9, 15e12
total_flops = 6 * N * D  # 6.3e24
peak = 1979e12 / 2  # "sparse" / 2
mfu = 0.5
per_day = peak * mfu * 1024 * 86400
days = total_flops / per_day  # 144
```

Five months on a thousand GPUs. Llama 3 70B took about that, at a lower MFU.

What is the largest model you can train on 8 H100s with AdamW?

```
gpu_bytes = 8 * 80e9
# params 2 + grads 2 + m 4 + v 4
bytes_per_param = 2 + 2 + 4 + 4
n_params = gpu_bytes / bytes_per_param
# -> 4e10
```

40B parameters, an *upper bound*: activations are not counted, and they depend on batch size and sequence length.

Takeaway

Rough, but enough to decide what to buy, what to run and what to expect. Today is about where these numbers come from.

Tensors, bytes and precision

1. Tensors, bytes and precision
2. Counting FLOPs
3. Memory during training
4. Arithmetic intensity: why FLOPs are not time
5. What you need for Assignment 1
6. Wrap-up

A tensor is a buffer plus a view

- Storage: one contiguous 1-D buffer on some device.
- View: shape, **strides** (elements to skip per index step), offset.
- `view`, `transpose`, slicing, `[:, None]`: new views of the *same* buffer. Free.
- `.contiguous()`, `.reshape` of a non-contiguous tensor, `.to(device)`: copies. They cost bandwidth.




```
>>> x = torch.arange(6).view(2, 3)
>>> x.stride()
(3, 1)
>>> y = x.T            # a view, no copy
>>> y.stride(), y.is_contiguous()
((1, 3), False)
>>> y.view(6)          # RuntimeError
>>> y.contiguous().view(6)  # copies
tensor([0, 3, 1, 4, 2, 5])
```

A "free" transpose followed by a matmul may trigger a hidden copy of a 10 GB tensor.

Floating point: fp32, and what a tensor costs

- Tensor elements are floating-point numbers. **fp32** (float32, single precision) is the default: the baseline of scientific computing, which reaches for fp64 when it matters. Deep learning can be less precise.
- Memory of a tensor
  = number of values × bytes per value.
- A matrix of GPT-3's feedforward layer ($4d_{\text{model}} \times d_{\text{model}}$, $d_{\text{model}}{=}12{,}288$): 2.3 GB.
  All of Llama 3 405B: 1.6 TB.
  While a single H100 has only 80 GB.

[fp32 on Wikipedia ↗]

```python
def mem(t):
    return t.numel() * t.element_size()

x = torch.zeros(4, 8)
assert x.dtype == torch.float32  # default
assert x.numel() == 4 * 8
assert x.element_size() == 4     # bytes
assert mem(x) == 4 * 8 * 4       # 128 B

# one matrix of GPT-3's MLP, in fp32
W = torch.empty(12288 * 4, 12288)
assert mem(W) == 2304 * 2**20    # 2.3 GB
```

fp16 halves the memory; bf16 keeps the range



sign · exponent = range · mantissa = resolution

fp32's range: fp16 no, bf16 yes

- **fp16** (float16, half precision): 2 bytes, half the memory.
- But 5 exponent bits: 10^{-8} underflows to 0, the smallest normal number is $6 \cdot 10^{-5}$. Small gradients vanish; hence loss scaling.

```
x = torch.zeros(4, 8, dtype=torch.half)
assert x.element_size() == 2
y = torch.tensor(1e-8, dtype=torch.half)
assert y == 0                    # underflow!
```

- **bf16**, brain float (Google Brain, 2018): the same 2 bytes, but fp32's 8 exponent bits and range.
- The catch: 7 mantissa bits, resolution $2^{-7} \approx 0.8\,\%$. Matters less than range in deep learning; the default since 2020.

```
bf16 = torch.bfloat16
z = torch.tensor(1e-8, dtype=bf16)
assert z != 0                    # no underflow
```

[fp16 ↗] [bf16 ↗] fp8 and int8/int4 follow the same logic (**L7**).

Try it now (1): precision

```
>>> import torch; bf16 = torch.bfloat16
>>> torch.finfo(bf16).eps
0.0078125                       # 2**-7
>>> torch.tensor(1 + 2**-8, dtype=bf16)
tensor(1., dtype=torch.bfloat16)   # lost
>>> torch.tensor(1 + 2**-7, dtype=bf16)
tensor(1.0078, dtype=torch.bfloat16)
>>> torch.tensor(70000., dtype=torch.float16)
tensor(inf, dtype=torch.float16)   # overflow
>>> x = torch.zeros(1000, 1000)
>>> x.numel() * x.element_size()
4000000                         # 4 MB
```

While it runs

- Learning rate 10^{-4} times gradient 10^{-3}: an update of 10^{-7}. Add it to a weight of size 1 in bf16. What happens?
- Hence: why keep an **fp32 master copy** of the weights when the forward runs in bf16?
- Sum 10^6 values of 10^{-3} in bf16 and in fp32.

First answer: nothing happens. The update is below bf16's resolution.

Mixed precision: the standard recipe

Implications on training:

- Training with fp32 works, but requires lots of memory.
- Training with fp16 and even bf16 is risky, and you can get instability.

1. **Master weights** and **optimizer states** in fp32.
2. Run the forward and backward matmuls in bf16; activations and gradients flow in bf16. Tensor cores accumulate in fp32 internally.
3. Compute the delicate parts in fp32: softmax, the loss, norm statistics, the optimizer update; apply the update to the master weights.

PyTorch has an automatic mixed prec. library: `torch.autocast(dev, dtype=torch.bfloat16)` around the forward does 2–3; a model kept in fp32 does the rest.

- Training state per parameter: bf16 copy (2) + bf16 grad (2) + fp32 master (4) + Adam m, v (8) $= \mathbf{16}$ bytes [Rajbhandari et al., 2020].
- bf16 tensor cores run $\sim 16\times$ faster than fp32 CUDA cores; the casts are nearly free.
- fp8 (DeepSeek-V3) adds per-block scaling so that 3 mantissa bits suffice. Still the exception.

Naming dimensions: `einsum` and `einops`

Einops is a library for manipulating tensors where dimensions are named, inspired by Einstein summation notation [Einstein, 1916]. [einops tutorial ↗]

```python
from einops import rearrange, einsum

# attention scores: no transposes, no guessing
S = einsum(q, k, "b h q d, b h k d -> b h q k")

# split heads without view/transpose gymnastics
q = rearrange(q, "b s (h d) -> b h s d", h=n_heads)

# a linear layer with any leading dims
y = einsum(x, W, "... i, o i -> ... o")
```

Named-axis type hints, `Float[Tensor, "batch seq d_model"]` (jaxtyping), catch shape bugs before they cost a GPU-hour.

- `einsum` lowers to the same matmul kernels; no speed cost.
- A1 convention: a linear layer stores $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, applied as in the last line.
- `rearrange`, `reduce`, `repeat` replace `view`, `permute`, `mean`, `expand` [Rogozhnikov, 2022].

Counting FLOPs

1. Tensors, bytes and precision
2. Counting FLOPs
3. Memory during training
4. Arithmetic intensity: why FLOPs are not time
5. What you need for Assignment 1
6. Wrap-up

The unit, and the speed of the machines

Two very confusing acronyms.

- 1 FLOP = one floating-point add or multiply; a multiply-add is **2 FLOPs**.
- FLOP/s is a rate: "$3.8 \cdot 10^{25}$ FLOPs" is a quantity, "$1 \cdot 10^{15}$ FLOP/s" a speed.
- Peak numbers assume dense bf16 tensor-core matmuls with perfect data movement. Nobody achieves them; the fraction achieved is the MFU.

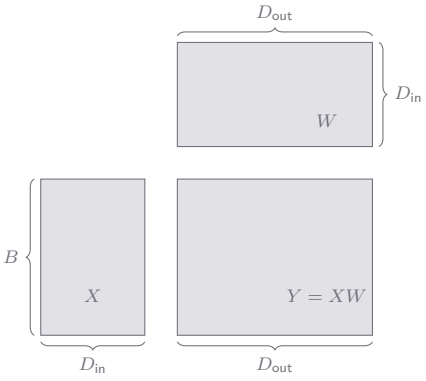
From FLOPs to wall-clock time

time = FLOPs needed / FLOP/s achieved

Device	bf16 peak TFLOP/s	fp32 peak TFLOP/s	HBM TB/s
Laptop CPU (8 cores)	~ 1	~ 0.5	0.05
NVIDIA T4 (Colab free)	65	8	0.3
NVIDIA A100 80 GB	312	19.5	2.0
NVIDIA H100 SXM	989	67	3.35
AMD MI250X (per GCD)	191	24	1.6

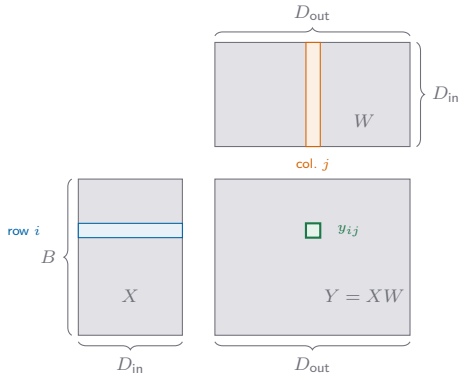
Datasheet numbers, dense, without the $2\times$ sparsity marketing. Assignments run on Colab: T4 (free; fp16 only, no bf16), L4 or A100 with Pro.

A matrix multiplication, counted



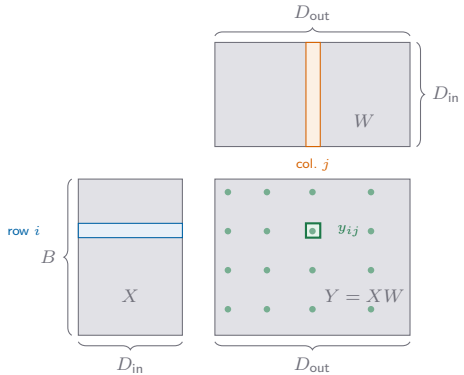
- $Y = XW$ with $X \in \mathbb{R}^{B \times D_{\text{in}}}$ (one row per token) and $W \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}$ (the weights).

A matrix multiplication, counted



- $Y = XW$ with $X \in \mathbb{R}^{B \times D_{\text{in}}}$ (one row per token) and $W \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}$ (the weights).
- One output $y_{ij} = \sum_{k=1}^{D_{\text{in}}} x_{ik}\, w_{kj}$ is a dot product of row i with column j: D_{in} multiplies and D_{in} adds, $\mathbf{2D}_{\text{in}}$ FLOPs.

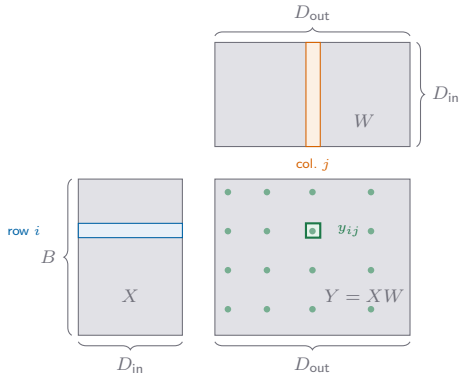
A matrix multiplication, counted



- $Y = XW$ with $X \in \mathbb{R}^{B \times D_{\text{in}}}$ (one row per token) and $W \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}$ (the weights).
- One output $y_{ij} = \sum_{k=1}^{D_{\text{in}}} x_{ik}\, w_{kj}$ is a dot product of row i with column j: D_{in} multiplies and D_{in} adds, $\mathbf{2D}_{\text{in}}$ FLOPs.
- There are $B \cdot D_{\text{out}}$ outputs. Total:

$$\text{FLOPs} = 2 \cdot B \cdot D_{\text{in}} \cdot D_{\text{out}}$$

A matrix multiplication, counted

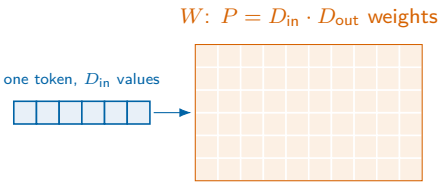


- $Y = XW$ with $X \in \mathbb{R}^{B \times D_{\text{in}}}$ (one row per token) and $W \in \mathbb{R}^{D_{\text{in}} \times D_{\text{out}}}$ (the weights).
- One output $y_{ij} = \sum_{k=1}^{D_{\text{in}}} x_{ik}\, w_{kj}$ is a dot product of row i with column j: D_{in} multiplies and D_{in} adds, $\mathbf{2D}_{\text{in}}$ FLOPs.
- There are $B \cdot D_{\text{out}}$ outputs. Total:

$$\text{FLOPs} = 2 \cdot B \cdot D_{\text{in}} \cdot D_{\text{out}}$$

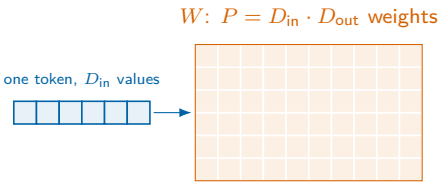
- Example: $B = 4096$ tokens through $W \in \mathbb{R}^{4096 \times 16384}$:
  $2 \cdot 4096 \cdot 4096 \cdot 16{,}384 = 5.5 \cdot 10^{11}$ FLOPs, about half a millisecond at H100 peak.

Read per parameter: 2 FLOPs per weight per token



- $2BD_{\text{in}}D_{\text{out}} = 2 \cdot P \cdot B$: every token passes through all P weights.

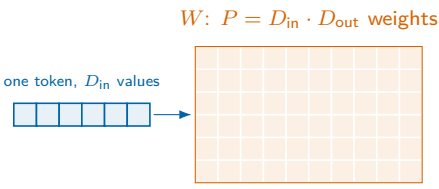
Read per parameter: 2 FLOPs per weight per token



every weight is used **once per token**: one multiply, one add

- $2BD_{\text{in}}D_{\text{out}} = 2 \cdot P \cdot B$: every token passes through all P weights.
- A model with N matmul parameters costs $\approx 2N$ FLOPs per token forward, whatever the architecture.

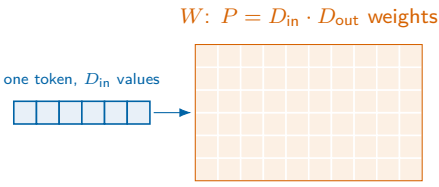
Read per parameter: 2 FLOPs per weight per token



every weight is used **once per token**: one multiply, one add
B **tokens** $\Rightarrow 2 \cdot P \cdot B$ **FLOPs**

- $2BD_{\text{in}}D_{\text{out}} = 2 \cdot P \cdot B$: every token passes through all P weights.
- A model with N matmul parameters costs $\approx 2N$ FLOPs per token forward, whatever the architecture.
- B makes the matmul worth it: the same weights serve many tokens. $B = 1$ (decoding) is the worst case; section 4.

Read per parameter: 2 FLOPs per weight per token



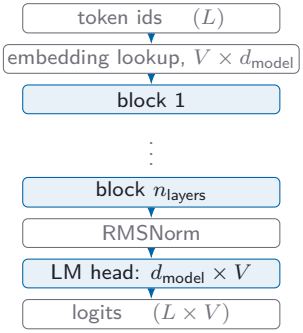
every weight is used **once per token**: one multiply, one add B **tokens** $\Rightarrow 2 \cdot P \cdot B$ **FLOPs**

- $2BD_{\text{in}}D_{\text{out}} = 2 \cdot P \cdot B$: every token passes through all P weights.
- A model with N matmul parameters costs $\approx 2N$ FLOPs per token forward, whatever the architecture.
- B makes the matmul worth it: the same weights serve many tokens. $B = 1$ (decoding) is the worst case; section 4.

Takeaway

Almost all Transformer FLOPs are matmuls: 2 FLOPs per parameter per token. Elementwise ops are negligible in FLOPs, not in time.

The Transformer, as a list of matrix multiplications



Blue boxes contain matmuls. The residual stream, one $L \times d_{\text{model}}$ matrix, runs through all of them.

Inside one block, input $x \in \mathbb{R}^{L \times d_{\text{model}}}$; blue = a matmul:

1. RMSNorm $\to \tilde{x}$ (no matmul)
2. $q = \tilde{x}W_Q, \quad k = \tilde{x}W_K, \quad v = \tilde{x}W_V,$ each $W \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$
3. split into n_{heads} heads of width $d_{\text{model}}/n_{\text{heads}}$; RoPE on q, k (no parameters)
4. $S = qk^\top / \sqrt{d_{\text{model}}/n_{\text{heads}}}$ per head, $S \in \mathbb{R}^{L \times L}$
5. $P = \text{softmax}(S + \text{causal mask})$ row by row (no matmul)
6. $o = Pv$ per head, concatenate the heads: $o \in \mathbb{R}^{L \times d_{\text{model}}}$
7. $x \leftarrow x + oW_O$ with $W_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$
8. RMSNorm $\to \tilde{x}$
9. $x \leftarrow x + \left(\text{SiLU}(\tilde{x}W_1) \odot \tilde{x}W_3\right) W_2,$ $W_1, W_3 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$

V vocabulary, L context, d_{model} width, n_{heads} heads, $d_{\text{ff}} \approx \frac{8}{3} d_{\text{model}}$, n_{layers} blocks. The *why* is Lecture 3; today only the shapes matter.

Forward pass of one Transformer block, per token

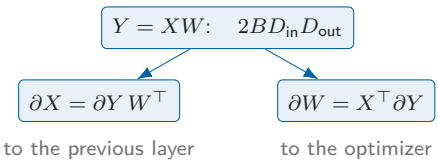
Sub-layer	Matrices	Parameters	FLOPs per token
Q, K, V, O projections	$4 \times (d_{\text{model}} \times d_{\text{model}})$	$4d_{\text{model}}^2$	$8d_{\text{model}}^2$
scores $QK^\top$	–	0	$2Ld_{\text{model}}$
weighted sum AV	–	0	$2Ld_{\text{model}}$
MLP (SwiGLU, $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$)	$3 \times (d_{\text{model}} \times d_{\text{ff}})$	$8d_{\text{model}}^2$	$16d_{\text{model}}^2$
RMSNorm $\times 2$, residuals	–	$2d_{\text{model}}$	$O(d_{\text{model}})$
per block		$12d_{\text{model}}^2$	$24d_{\text{model}}^2 + 4Ld_{\text{model}}$

- Whole model, per token: $n_{\text{layers}}(24d_{\text{model}}^2 + 4Ld_{\text{model}}) + 2d_{\text{model}}V$ (the LM head).
- With $N = 12\, n_{\text{layers}} d_{\text{model}}^2$ block parameters: forward $\approx \mathbf{2N} + 4n_{\text{layers}}Ld_{\text{model}} + 2d_{\text{model}}V$.

- The scores are the only term in L: each query meets L keys of total size d_{model}.
- GQA shrinks K, V; MoE replaces $8d_{\text{model}}^2$ by the *active* experts (**L3**). Embedding lookup: ≈ 0 FLOPs.

Backward pass: twice the forward

For every matmul $Y = XW$ of the forward pass, the backward pass needs two matmuls of the same size:



- Backward $\approx 2\times$ forward; a training step $\approx 3\times$ forward.
- Per parameter per token: 2 (fwd) $+4$ (bwd) $= \mathbf{6}$.

The $6ND$ rule

Training FLOPs $\approx 6 \cdot N \cdot D$:
N params in the matmuls, D train tokens.

[Kaplan et al., 2020; Hoffmann et al., 2022]

- The first layer needs no ∂X, the head is one matmul: rounding errors at scale.
- Activation recomputation (section 3) re-runs the forward: $\mathbf{8ND}$.
- D counts *processed* tokens: two epochs over 1T tokens is $D = 2\text{T}$.

When $6ND$ is wrong, and by how much

Situation	What changes	Size of the effect
Long context	add $3 \cdot 4 n_{\text{layers}} L d_{\text{model}}$ per token for the scores	$L/6d_{\text{model}}$: $+33\,\%$ at $L = 8$k, $d_{\text{model}} = 4$k; $5\times$ at 128k
Big vocabulary, small model	add $3 \cdot 2 d_{\text{model}} V$ per token for the head	A1: $+27\,\%$; Llama 3 8B: $+2\,\%$
Mixture of experts	N is the *active* parameter count	DeepSeek-V3: $N = 37$B, not 671B
Activation recomputation	one extra forward pass	$8ND$ instead of $6ND$
Embedding parameters	in "N" of press releases, ≈ 0 FLOPs	A1: 45 % of the parameters

Takeaway

$6ND$ is 10 %-accurate for dense models at moderate context if N excludes embeddings and D counts tokens *seen* (epochs included). Otherwise this table gives approximate correction.

Worked example: the A1 reference model

$V = 10{,}000$, $d_{\text{model}} = 512$, $n_{\text{layers}} = 4$, $d_{\text{ff}} = 1344$, $L = 256$, untied embeddings.

(4 blocks)	Parameters	Fwd FLOPs / token
Token embedding	5,120,000	0
Attention projections	4,194,304	8,388,608
SwiGLU MLPs	8,257,536	16,515,072
Scores and AV	0	2,097,152
Norms	4,608	≈ 0
LM head	5,120,000	10,240,000
Total	22,696,448	37,240,832

- Training: $3 \times 3.7 \cdot 10^7 = 1.1 \cdot 10^8$ FLOPs per token; $D = 2 \cdot 10^8$: $2.2 \cdot 10^{16}$ FLOPs.
- Naive $6ND$: $2.7 \cdot 10^{16}$ with embeddings counted, $1.5 \cdot 10^{16}$ with the head forgotten. Both within $2\times$.
- A100 at 40 % MFU: **3 minutes**; at a realistic 20 %, 6 minutes; a Colab T4 at 20 % in fp16: 30 minutes. Cheap: you will train it many times.

The head is 27 % of the forward FLOPs. Lecture 3 asks whether to tie it to the embedding.

MFU: how much of the machine you actually use

Model FLOPs utilisation [Chowdhery et al., 2022]

FLOPs per token, times tokens per second, divided by the peak FLOP/s of the hardware.

- Counts only *useful* FLOPs, $6N$ per token; recomputed activations do not count.
- Measure it in every run; tokens/s is one line of logging.
- Typical: 35–45 % for large dense models on H100 clusters (Llama 3: 38–43 %); 10–30 % for small models on one GPU.

Where the other 60 % goes

- Memory-bound kernels: norms, softmax, residual adds, the optimizer step (**L4**).
- Kernel launches and Python between kernels; small matrices.
- Communication and pipeline bubbles (**L5**); data loading, logging.

Key idea

MFU is the efficiency score of this course. A2 reports it for every run.

Try it now (2): the speed of your machine

```
import torch, time
n, dt = 4096, torch.float16     # bf16 on A100+; CPU:
    1024, fp32
a = torch.randn(n, n, device="cuda", dtype=dt)
b = torch.randn(n, n, device="cuda", dtype=dt)
for _ in range(3): a @ b        # warm-up
torch.cuda.synchronize()
t = time.perf_counter()
for _ in range(20): c = a @ b
torch.cuda.synchronize()
sec = (time.perf_counter() - t) / 20
print(f"{2 * n**3 / sec / 1e12:.1f} TFLOP/s")
```

What to expect

- Laptop CPU, fp32: 0.1–1 TFLOP/s.
- Colab T4, fp16: 20–30.
- A100: 250–290 of 312. H100: 600–750 of 989.

- Set $n = 256$: the number collapses. Launch overhead and memory traffic dominate.
- Try `a @ b[:, :1]`: a matrix-vector product, the decoding regime (**L7**).

Memory during training

1. Tensors, bytes and precision
2. Counting FLOPs
3. Memory during training
4. Arithmetic intensity: why FLOPs are not time
5. What you need for Assignment 1
6. Wrap-up

The four occupants of GPU memory

Pool	Size	Scales with	Lives
Parameters	$2N$ bf16 $+4N$ fp32 master	N	always
Gradients	$2N$ bf16 (or $4N$ fp32)	N	backward to step
Optimizer states	$8N$ (AdamW: m, v in fp32)	N	always
Activations	$\propto b \cdot L \cdot d_{\text{model}} \cdot n_{\text{layers}}$	batch, L, N	forward to backward

- Static state: **16 bytes per parameter** with mixed precision, and 16 with plain fp32 too $(4 + 4 + 8)$.
- Llama 3 405B: $16 \times 4.05 \cdot 10^{11} = 6.5\,\text{TB}$ of state before the first activation; 81 GPUs of 80 GB just to *hold* it. Hence sharding (**L5**).

Two kinds of memory

Static pools scale with the *model*, activations with the *batch*. Activations trade against compute, static state against communication.

Measure, don't guess: `torch.cuda.max_memory_allocated()` after one step; `memory_summary()` when it dies.

Activations: the pool that scales with the batch

- The backward pass needs what the forward saw: inputs of every matmul, attention probabilities, norm statistics.
- Per block, bf16, without FlashAttention [Korthikanti et al., 2022]: bytes $\approx s\,b\,h\,(34 + 5\,a\,s/h)$ with s sequence length, b batch, $h = d_{\text{model}}$, a heads. The 34 collects matmul inputs; $5as/h$ is the $L \times L$ attention matrix per head.
- FlashAttention (**L4**) never materialises that matrix: the second term disappears.
- **Activation checkpointing**: keep only block inputs, recompute the rest during backward. Memory $\div \sim 10$, compute $\times 1.33$.

A1 model, $b = 32$, $s = 256$, $h = 512$, $a = 16$:

- $34\,sbh = 143$ MB per block
- $5as/h = 40 \Rightarrow 168$ MB per block
- 4 blocks: ≈ 1.2 GB. Any GPU, even the laptop.

Llama 3 405B, $h = 16384$, 126 blocks, one 8k sequence: $34 \cdot 8192 \cdot 16384 \cdot 126 = 575$ GB without the attention matrices. Not without checkpointing and sharding.

Batch size, in tokens

- The batch size that matters is **tokens per optimizer step**, $b \times s$. GPT-3: 3.2M. Llama 3 405B: 4M, later 16M. Your A1 run: 8k–64k.
- Larger batches: matmuls with more rows, fewer steps, less communication per token. But beyond the **critical batch size**, extra tokens per step stop reducing the number of steps needed (**L6**).
- **Gradient accumulation**: run k micro-batches, sum the gradients, step once. Identical mathematics, $k\times$ less activation memory, same FLOPs.

Takeaway

Memory decides the *micro*-batch; the optimisation problem decides the *global* batch. Accumulation decouples the two.

A1 default: $b = 32$, $s = 256$, so 8 192 tokens per step; $2 \cdot 10^8$ tokens is 24k steps. At 3 minutes total that is 7 ms per step; a Python loop over 4 blocks can just about do it.

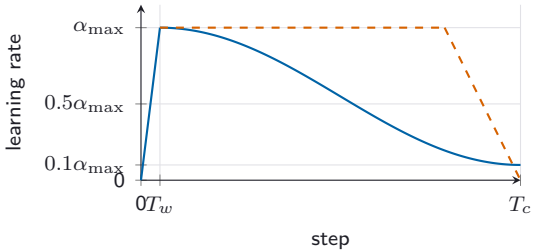
The optimizer: AdamW

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t, \qquad \hat{m}_t = m_t/(1-\beta_1^t)$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2, \qquad \hat{v}_t = v_t/(1-\beta_2^t)$$
$$\theta_t = \theta_{t-1} - \alpha_t\Big(\hat{m}_t/(\sqrt{\hat{v}_t}+\epsilon) + \lambda\,\theta_{t-1}\Big)$$

- m: momentum; v: per-parameter scale. Updates have unit size in every coordinate whatever the gradient magnitude, which varies by orders of magnitude across a Transformer.

- Weight decay is *decoupled*, not added to g [Loshchilov & Hutter, 2019]. LM defaults: $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\lambda = 0.1$.
- Memory: m, v in fp32, 8 bytes per parameter, half of the 16. Compute: $\sim 1\,\%$ of the step.
- Cheaper: 8-bit Adam [Dettmers et al., 2022]. Muon and friends change the geometry, not the memory.
- **A1**: you implement it yourself, about 10 lines, and check it against `torch.optim.AdamW`.

Learning-rate schedule and gradient clipping



Blue: linear warmup, cosine decay to 10 % (GPT-3, Llama). Orange: warmup–stable–decay, popular since 2024 because the stable phase can be extended without restarting.

Warmup: $\alpha_t = \alpha_{\max}\, t/T_w$. Cosine: $\alpha_t = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min})(1 + \cos\phi_t)$ with $\phi_t = \pi \frac{t - T_w}{T_c - T_w}$.

- Warmup: Adam's statistics are noise for the first few hundred steps. Decay: most of the final loss improvement happens there.
- **Gradient clipping**: rescale g so that $\|g\|_2 \le 1$. Cheap insurance against loss spikes; log $\|g\|_2$ as a health signal.
- $\alpha_{\max}$ matters most. A1 asks for a sweep; **L6** shows how it transfers.

Memory budget of the A1 run, and of a 7B run

	A1 model, fp32, 8k tokens per step	7B, mixed precision, 64k tokens per step
Parameters	91 MB	14 GB bf16 + 28 GB fp32 master
Gradients	91 MB	14 GB
AdamW states	182 MB	56 GB
Activations	≈ 2.5 GB	32 blocks $\times$ 34 $\times$ 64k $\times$ 4096 $= 285$ GB
Total	≈ 3 GB	≈ 400 GB

- A1: memory is irrelevant, **time** is the constraint. Use bf16 autocast anyway, to learn the workflow.
- 7B: five H100s of state and activations for a modest batch. Checkpointing brings activations to ~ 30 GB; ZeRO/FSDP spreads the 112 GB of state over the data-parallel GPUs (**L5**).

Reading an out-of-memory error

At the first forward: activations, halve the micro-batch. At the optimizer step: static state, shard it. After hours: a leak or fragmentation, read `memory_summary()`.

Arithmetic intensity: why FLOPs are not time

1. Tensors, bytes and precision
2. Counting FLOPs
3. Memory during training
4. Arithmetic intensity: why FLOPs are not time
5. What you need for Assignment 1
6. Wrap-up

Arithmetic intensity: five computations, one question

To compute: *move the inputs to the accelerator, compute, move the outputs back.*

With perfect overlap, **time** $=$ **max(bytes / bandwidth, FLOPs / peak)**.

Example: H100, bf16: $989 \cdot 10^{12} / 3.35 \cdot 10^{12} = \mathbf{295}$ FLOPs per byte is the break-even.

Computation	bytes moved (bf16)	FLOPs	I	vs. 295

Arithmetic intensity: five computations, one question

To compute: *move the inputs to the accelerator, compute, move the outputs back.*

With perfect overlap, **time = max(bytes / bandwidth, FLOPs / peak)**.

Example: H100, bf16: $989 \cdot 10^{12} / 3.35 \cdot 10^{12} = \mathbf{295}$ FLOPs per byte is the break-even.

Computation	bytes moved (bf16)	FLOPs	I	vs. 295
ReLU of $n = 2^{20}$ values	read $2n$, write $2n$	n	0.25	memory-bound

Arithmetic intensity: five computations, one question

To compute: *move the inputs to the accelerator, compute, move the outputs back.*

With perfect overlap, **time = max(bytes / bandwidth, FLOPs / peak)**.

Example: H100, bf16: $989 \cdot 10^{12} / 3.35 \cdot 10^{12} = \mathbf{295}$ FLOPs per byte is the break-even.

Computation	bytes moved (bf16)	FLOPs	I	vs. 295
ReLU of $n = 2^{20}$ values	read $2n$, write $2n$	n	0.25	memory-bound
GELU of n values	$4n$	$\approx 20n$	5	memory-bound

- GELU does $20\times$ the work of ReLU per byte and takes *exactly as long*: in isolation, ReLU is not faster.

Arithmetic intensity: five computations, one question

To compute: *move the inputs to the accelerator, compute, move the outputs back.*

With perfect overlap, **time = max(bytes / bandwidth, FLOPs / peak)**.

Example: H100, bf16: $989 \cdot 10^{12} / 3.35 \cdot 10^{12} = \mathbf{295}$ FLOPs per byte is the break-even.

Computation	bytes moved (bf16)	FLOPs	I	vs. 295
ReLU of $n = 2^{20}$ values	read $2n$, write $2n$	n	0.25	memory-bound
GELU of n values	$4n$	$\approx 20n$	5	memory-bound
dot product, n values	$2n + 2n + 2$	$2n - 1$	0.5	memory-bound

- GELU does $20\times$ the work of ReLU per byte and takes *exactly as long*: in isolation, ReLU is not faster.

Arithmetic intensity: five computations, one question

To compute: *move the inputs to the accelerator, compute, move the outputs back.*

With perfect overlap, **time = max(bytes / bandwidth, FLOPs / peak)**.

Example: H100, bf16: $989 \cdot 10^{12} / 3.35 \cdot 10^{12} = \mathbf{295}$ FLOPs per byte is the break-even.

Computation	bytes moved (bf16)	FLOPs	I	vs. 295
ReLU of $n = 2^{20}$ values	read $2n$, write $2n$	n	0.25	memory-bound
GELU of n values	$4n$	$\approx 20n$	5	memory-bound
dot product, n values	$2n + 2n + 2$	$2n - 1$	0.5	memory-bound
matrix–vector xW, $n = 1024$	$2n + 2n^2 + 2n$	$n(2n - 1)$	≈ 1	memory-bound

- GELU does $20\times$ the work of ReLU per byte and takes *exactly as long*: in isolation, ReLU is not faster.
- A matrix–vector product is what decoding does, one token at a time: $300\times$ below peak.

Arithmetic intensity: five computations, one question

To compute: *move the inputs to the accelerator, compute, move the outputs back.*

With perfect overlap, **time = max(bytes / bandwidth, FLOPs / peak)**.

Example: H100, bf16: $989 \cdot 10^{12} / 3.35 \cdot 10^{12} = \mathbf{295}$ FLOPs per byte is the break-even.

Computation	bytes moved (bf16)	FLOPs	I	vs. 295
ReLU of $n = 2^{20}$ values	read $2n$, write $2n$	n	0.25	memory-bound
GELU of n values	$4n$	$\approx 20n$	5	memory-bound
dot product, n values	$2n + 2n + 2$	$2n - 1$	0.5	memory-bound
matrix–vector xW, $n = 1024$	$2n + 2n^2 + 2n$	$n(2n - 1)$	≈ 1	memory-bound
matmul XW, $n = 1024$	$3 \cdot 2n^2$	$n^2(2n - 1)$	$n/3 = 341$	**compute-bound**

- GELU does $20\times$ the work of ReLU per byte and takes *exactly as long*: in isolation, ReLU is not faster.
- A matrix–vector product is what decoding does, one token at a time: $300\times$ below peak.
- Only the matmul crosses the line, because $I \approx n/3$ grows with the matrix. Training is big matmuls.

Arithmetic intensity: five computations, one question

To compute: *move the inputs to the accelerator, compute, move the outputs back.*

With perfect overlap, **time = max(bytes / bandwidth, FLOPs / peak)**.

Example: H100, bf16: $989 \cdot 10^{12} / 3.35 \cdot 10^{12} = \mathbf{295}$ FLOPs per byte is the break-even.

Computation	bytes moved (bf16)	FLOPs	I	vs. 295
ReLU of $n = 2^{20}$ values	read $2n$, write $2n$	n	0.25	memory-bound
GELU of n values	$4n$	$\approx 20n$	5	memory-bound
dot product, n values	$2n + 2n + 2$	$2n - 1$	0.5	memory-bound
matrix–vector xW, $n = 1024$	$2n + 2n^2 + 2n$	$n(2n - 1)$	≈ 1	memory-bound
matmul XW, $n = 1024$	$3 \cdot 2n^2$	$n^2(2n - 1)$	$n/3 = 341$	**compute-bound**

- GELU does $20\times$ the work of ReLU per byte and takes *exactly as long*: in isolation, ReLU is not faster.
- A matrix–vector product is what decoding does, one token at a time: $300\times$ below peak.
- Only the matmul crosses the line, because $I \approx n/3$ grows with the matrix. Training is big matmuls.
- In general we find ourselves memory-bound. Precision matters too: fp32 doubles the bytes and halves I.

Arithmetic intensity: what follows for training and serving

Definition

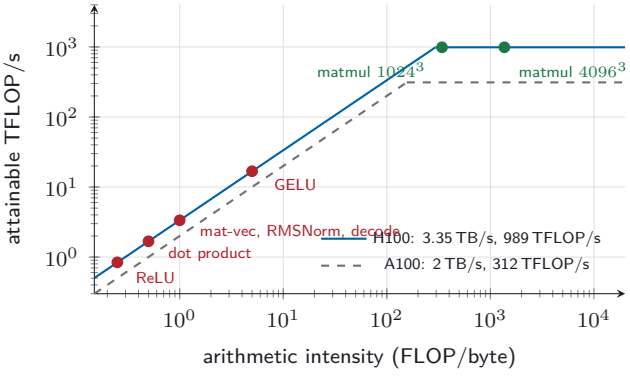
$I =$ FLOPs / bytes moved. Compute-bound if I exceeds the ridge point peak FLOP/s / bandwidth: 156 on an A100, 295 on an H100.

- Matmul XW, B rows, $D \times D$ weights, bf16: $2BD^2$ FLOPs, $2(2BD + D^2)$ bytes. For $D \gg B$: $I \approx B$. **The intensity of a matmul is its batch size.**
- RMSNorm on $L \times d_{\text{model}}$: $\sim 4Ld_{\text{model}}$ FLOPs, $4Ld_{\text{model}}$ bytes, $I \approx 1$. Softmax, residual adds: the same. Memory-bound, $300\times$ below peak.

Consequences

- Training, thousands of tokens per matmul: compute-bound.
- Decoding at $B = 1$: every weight is read once for 2 FLOPs. Memory-bound by $300\times$; that is **L7** in one line.
- Fusing memory-bound ops into one kernel (FlashAttention, fused norms, `torch.compile`) is where single-GPU speedups come from (**L4**).

The roofline in one picture



Takeaway

Count bytes as carefully as FLOPs. Time is the maximum of the two, not the sum. [Williams et al., 2009]

- A training step is a mix: big matmuls on the roof, dozens of small kernels on the floor. The average is the MFU.
- Every optimisation in **L4** moves a point right (fusion, bigger tiles) or up (better kernels).

Estimating the cost of a run, start to finish

1. Count N (matmul parameters) and D (tokens): $6ND$ plus corrections.
2. Pick hardware and an honest MFU: 40 % for a large run, 20 % for a small one.
3. GPU-hours $=$ FLOPs$/($peak $\times$ MFU $\times\, 3600)$.
4. Price: 2–4 EUR per H100-hour on the market; Colab Pro budgets "compute units".
5. Add 30–100 % for failed runs, sweeps and evaluation. Nobody trains once.

Run	FLOPs	GPU-h	
A1 model	$2.2 \cdot 10^{16}$	0.05	one A100
A2 sweep	$\sim 1 \cdot 10^{17}$	3–6	T4, per group
Llama 3 8B	$7.2 \cdot 10^{23}$	$5 \cdot 10^{5}$	15T tokens
Llama 3 405B	$3.8 \cdot 10^{25}$	$2.7 \cdot 10^{7}$	68 days

Sanity check, last row: $3.96 \cdot 10^{14}/(6 \cdot 4.05 \cdot 10^{11}) = 163$ tokens/s per GPU; $\times 16{,}384$ GPUs $= 2.7 \cdot 10^{6}$ tokens/s; $1.56 \cdot 10^{13}$ tokens take 68 days. The report says about two months.

Takeaway

Tokens/s per GPU $=$ MFU $\times$ peak$/6N$. Check every paper's numbers with this line.

What you need for Assignment 1

1. Tensors, bytes and precision
2. Counting FLOPs
3. Memory during training
4. Arithmetic intensity: why FLOPs are not time
5. What you need for Assignment 1
6. Wrap-up

The training loop you will write

```
data = np.memmap("train.bin", np.uint16, "r")
model = TransformerLM(cfg).to(dev)
opt = AdamW(model.parameters(), lr=cfg.lr_max,
            betas=(0.9, 0.95), weight_decay=0.1)
for step in range(start, cfg.steps):
    x, y = get_batch(data, cfg.batch, cfg.ctx, dev)
    set_lr(opt, lr_cosine(step, cfg))
    with torch.autocast(dev, dtype=torch.bfloat16):
        logits = model(x)
    loss = cross_entropy(logits.float(), y)
    loss.backward()
    gnorm = clip_grad_norm_(model.parameters(), 1.0)
    opt.step(); opt.zero_grad(set_to_none=True)
    if step % 100 == 0: log(step, loss, gnorm, tok_s)
    if step % 2000 == 0: save_ckpt(model, opt, step)
```

- Every line is a function you implement; the tests check each in isolation (`adapters.py`).
- No epochs: sample random windows from the token stream (next slide).
- Log **tokens/s** and **gradient norm** from the first run on: your MFU and your health monitor.

Data loading and the loss

Data

- Tokenize once; store ids as `uint16` (fits $V \leq 65535$), 2 bytes per token. TinyStories: ≈ 0.5B tokens, 1 GB. Memory-map it, never read it into RAM.
- A batch: b random offsets i; inputs `ids[i:i+L]`, targets `ids[i+1:i+L+1]`. Windows may straddle `<|endoftext|>`; that is fine.
- Validation split: a separate file, never sampled for training.

Cross-entropy, done stably

```
def cross_entropy(logits, targets):
    # fp32 logits (..., V); subtract the max
    m = logits.max(-1, keepdim=True)[0]
    z = logits - m
    lse = z.exp().sum(-1).log()
    tgt = z.gather(-1, targets[..., None])
    return (lse - tgt[..., 0]).mean()
```

- Initial loss must be $\approx \ln V = 9.2$ for $V = 10$k. If not, the bug is in the head or the init.
- Logits leave autocast in bf16: upcast before the loss.

Checkpointing, resuming, and the debugging checklist

Checkpoints

- Save `model.state_dict()`, `opt.state_dict()` and `step` together; the optimizer state is $2\times$ the model.
- Resuming must reproduce the schedule: the learning rate is a function of `step`, not of wall-clock time.
- Colab sessions time out and disconnect. A2 runs are *resumed*, always.

Report for A1: loss curves, tokens/s and MFU, a learning-rate sweep, samples.

Before you train for real

1. Initial loss $\approx \ln V$.
2. Overfit one batch: loss $\to 0$ in a few hundred steps. If not, the model or the optimizer is broken.
3. Gradient norm finite and roughly stable.
4. Tokens/s $\times$ FLOPs per token matches the expected MFU within $2\times$. If not, find the memory- or Python-bound part.
5. Sample. Garbage after 1 000 steps is normal, after 10 000 it is a bug.

Wrap-up

1. Tensors, bytes and precision
2. Counting FLOPs
3. Memory during training
4. Arithmetic intensity: why FLOPs are not time
5. What you need for Assignment 1
6. Wrap-up

Summary

1. Bytes: elements $\times$ bytes per element; bf16 for matmuls, fp32 where the numbers are delicate; **16 bytes per parameter** of training state.
2. FLOPs: 2 per parameter per token forward, 6 for training: $\mathbf{6ND}$, plus a short table of corrections (long context, big vocabulary, MoE, recomputation).
3. Memory: four pools. Parameters, gradients and optimizer states scale with the model; activations scale with the batch and trade against compute.
4. Time is not FLOPs: arithmetic intensity decides compute-bound vs memory-bound; **the intensity of a matmul is its batch size**; MFU is the score.
5. You can now cost any run in a paper from two numbers, and you have the skeleton of the A1 training loop.

The two opening questions: 144 days ($6ND$ over $1024\times$ peak $\times$ MFU), and 40B parameters (640 GB over 16 bytes each), before activations.

Next: Lecture 3 – Architectures

Thursday 8 October, Seminarraum 6

Coming up

- The modern Transformer block: pre-norm, RMSNorm, RoPE, SwiGLU, GQA
- Hyperparameters that matter, and the ones that do not
- Stability: warmup, z-loss, QK-norm
- Attention variants and mixture of experts
- How to read a model card

Reading

- Kaplan et al. (2020), section 2.1: parameter and compute counts
- Korthikanti et al. (2022), section 4: activation memory
- kipply, Transformer inference arithmetic (blog)

Do

- Finish the tokenizer; implement `cross_entropy` and `AdamW`; run today's scripts on any GPU you can reach.

References

- Kaplan et al. (2020). Scaling laws for neural language models. arXiv:2001.08361.
- Hoffmann et al. (2022). Training compute-optimal large language models (Chinchilla). arXiv:2203.15556.
- Chowdhery et al. (2022). PaLM: Scaling language modeling with Pathways (defines model FLOPs utilisation, MFU). arXiv:2204.02311.
- Korthikanti et al. (2022). Reducing activation recomputation in large Transformer models. arXiv:2205.05198.
- Rajbhandari, Rasley, Ruwase & He (2020). ZeRO: Memory optimizations toward training trillion parameter models. *SC*.
- Micikevicius et al. (2018). Mixed precision training. *ICLR*.
- Kingma & Ba (2015). Adam: A method for stochastic optimization. *ICLR*.
- Loshchilov & Hutter (2019). Decoupled weight decay regularization (AdamW). *ICLR*.
- Dettmers, Lewis, Shleifer & Zettlemoyer (2022). 8-bit optimizers via block-wise quantization. *ICLR*.
- Williams, Waterman & Patterson (2009). Roofline: An insightful visual performance model for multicore architectures. *CACM*.
- Rogozhnikov (2022). Einops: Clear and reliable tensor manipulations with Einstein-like notation. *ICLR*.
- Chen (kipply) (2022). Transformer inference arithmetic. `https://kipp.ly/transformer-inference-arithmetic/`.

References (cont.)

- Grattafiori et al. (2024). The Llama 3 herd of models. arXiv:2407.21783.
- DeepSeek-AI (2024). DeepSeek-V3 technical report. arXiv:2412.19437.
- NVIDIA (2022). H100 Tensor Core GPU datasheet. `https://www.nvidia.com/en-us/data-center/h100/`.